1. Introduction

Every time a calculator, spreadsheet or programming language works out an answer such as 3 + 5 * 2, it must decide which part to calculate first. A human does this naturally by remembering that multiplication comes before addition. A computer, however, reads the text one symbol at a time and needs a method to remember what it has seen and what it still has to do. One of the simplest and most reliable ways to do this is to use a data structure called a stack.

This assignment implements a Python program that reads mathematical expressions from a text file (input.txt), evaluates each expression using stacks, and writes the answers to another text file (output.txt) in the same layout as the input. The program supports the operators addition (+), subtraction (-), multiplication (*) and division (/), brackets, and the correct rules of precedence.

1.1 Objectives

- Create a Stack class with the operations push, pop and peek.
- Parse and evaluate expressions with operators, brackets and precedence rules.
- Read expressions from input.txt and write the results to output.txt, keeping the separator lines.
- Test the program with different expressions and document the results.

1.2 Tools Used

The program was written in Python 3 using only the standard library, so no extra packages need to be installed. Source code is stored on GitHub for version control and submission.

2. Data Structures Used

2.1 Description of the Stack Data Structure

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle: the last item added is the first item removed. A good real-life example is a pile of plates. A new plate is always placed on top of the pile, and when a plate is needed it is also taken from the top.

Operation	Meaning
push(item)	Adds an item to the top of the stack.
pop()	Removes the top item and returns it. Raises an error if the stack is empty.
peek()	Returns the top item without removing it. Raises an error if the stack is empty.
is_empty()	Returns True if the stack has no items.
size()	Returns the number of items in the stack.

In this program the Stack class stores its items in a Python list. The end of the list is treated as the top of the stack, so push uses list.append() and pop uses list.pop(). Both operations take constant time, O(1).

2.2 How the Stack Is Used to Evaluate Expressions

The program uses two stacks together. This is a simplified version of the well-known Shunting-Yard idea of Edsger Dijkstra.

- **Operand stack:** holds the numbers waiting to be used.
- **Operator stack:** holds operators (+, -, *, /) and opening brackets that are waiting for their turn.

The expression is first broken into tokens (numbers, operators and brackets). Each token is then handled as follows:

- A number is pushed onto the operand stack.
- An opening bracket "(" is pushed onto the operator stack.
- A closing bracket ")" makes the program apply operators until the matching "(" is found, then remove the "(".
- An operator first applies any operator already on top of the stack that has the same or higher precedence, and then is pushed onto the operator stack.
- When all tokens are read, any operators left on the stack are applied. The one number left on the operand stack is the final answer.

To apply an operator, the program pops one operator and two numbers, calculates the result and pushes the result back on the operand stack. Multiplication and division have precedence 2, and addition and subtraction have precedence 1.

2.3 Worked Example: 3 + 5 * 2

Step	Token read	Operand stack (top at right)	Operator stack (top at right)	Action
1	3	[3]	[ ]	Push number
2	+	[3]	[+]	Push operator
3	5	[3, 5]	[+]	Push number
4	*	[3, 5]	[+, *]	* is higher than +, so push
5	2	[3, 5, 2]	[+, *]	Push number
6	End	[3, 10]	[+]	Apply *: 5 * 2 = 10

Step	Token read	Operand stack (top at right)	Operator stack (top at right)	Action
7	End	[13]	[ ]	Apply +: 3 + 10 = 13

The final answer, 13, is correct because multiplication was performed before addition.

2.4 Importance of Stacks in Computing

Stacks are used in many areas of computing, which is why they are a core topic in data structures:

- Function calls: the call stack remembers where a program must return after a function ends.
- Undo and redo: text editors and design tools store previous actions on a stack.
- Expression evaluation and syntax checking: compilers and calculators use stacks to evaluate expressions and match brackets.
- Web browsers: the Back button uses a stack of visited pages.
- Depth-first search and backtracking: algorithms for mazes, puzzles and graphs rely on stacks.

3. Program Design

3.1 Architecture of the Program

The program is divided into three simple parts so that each part has one job:

Part	Main functions	Job
1. Stack	Stack class	Provides push, pop, peek, is_empty and size.
2. Evaluator	tokenize(), apply_operator(), evaluate(), format_result(), safe_evaluate()	Turns text into tokens and calculates the answer using two stacks.
3. File handling	is_separator(), process_file(), main()	Reads input.txt, calls the evaluator for each line and writes output.txt.

3.2 Description of Key Functions

Function	Description
Stack.push / pop / peek	The three required stack operations. pop and peek raise an IndexError if the stack is empty, which prevents the program from using missing data.
tokenize(expression)	Reads the text one character at a time, ignores spaces, groups digits and decimal points into numbers, and recognises operators and brackets. It also detects a negative sign (for example 2 * -3). Unknown characters raise a ValueError.
apply_operator(operands, operators)	Pops one operator and two numbers, performs the calculation and pushes the result. Division by zero raises a ZeroDivisionError.
evaluate(expression)	The main algorithm. Uses the operand stack and operator stack and the precedence rules to calculate the final answer.
format_result(value)	Displays whole numbers without a decimal point (13 instead of 13.0) and rounds decimals to four places.
safe_evaluate(expression)	Calls evaluate() but catches errors and returns a clear message such as "Error: invalid expression" so the program does not crash.
is_separator(line)	Returns True if a line contains only dashes (three or more), such as ----.
process_file(input, output)	Reads each line of input.txt. Separator lines are copied unchanged; expression lines are replaced by their results; blank lines are skipped. The output is written to output.txt.

3.3 Pseudocode

Pseudocode for the main evaluation algorithm:

```
FUNCTION evaluate(expression):
    operands  = new Stack()
    operators = new Stack()
    FOR each token in tokenize(expression):
        IF token is a number:
            operands.push(token)
        ELSE IF token is "(":
            operators.push(token)
```

```
        ELSE IF token is ")":
            WHILE operators.peek() is not "(":
                apply_operator()
            operators.pop()              // remove "("
        ELSE:                            // + - * /
            WHILE operators not empty AND top is not "("
                  AND precedence(top) >= precedence(token):
                apply_operator()
            operators.push(token)
    WHILE operators not empty:
        apply_operator()
    RETURN operands.pop()

FUNCTION apply_operator():
    op    = operators.pop()
    right = operands.pop()
    left  = operands.pop()
    operands.push(left op right)
```

3.4 Program Flow

The overall flow of the program is:

```
START
  |
  v
Open input.txt  -->  file missing? --> show message, STOP
  |
  v
Read next line  <------------------------------+
  |                                            |
  +-- blank line?      --> skip ---------------+
  +-- separator (---)? --> copy to output ------+
  +-- expression?      --> tokenize             |
                       --> evaluate (2 stacks)
                       --> save result -----+
  |
  v  (no more lines)
Write all results to output.txt
```

```
 |
 v
END
```

4. Testing and Results

4.1 How the Program Was Tested

The program was tested in two ways. First, the sample input.txt with three expressions was run to check that the output.txt file has the correct values and keeps the separator lines. Second, extra expressions were tested to check negative numbers, decimals, repeated operators, nested brackets and error situations. For every test, the expected answer was worked out by hand and compared with the program output.

4.2 Sample Input and Output

input.txt	output.txt
3 + 5 * 2	13
-----	-----
(8 / 4) + 7 * 2	16
-----	-----
10 - (2 + 3) * 4	-10

Note: the sample output printed in the assignment sheet shows 17 for the second expression. Working it out by hand gives (8 / 4) = 2, then 7 * 2 = 14, and 2 + 14 = 16. The program therefore outputs 16, which is mathematically correct. The value 17 on the sheet appears to be a typing error.

4.3 Additional Test Cases

No.	Expression	Expected	Program result	Status
1	3 + 5 * 2	13	13	Pass
2	(8 / 4) + 7 * 2	16	16	Pass
3	10 - (2 + 3) * 4	-10	-10	Pass
4	7 - 2 - 1	4	4	Pass
5	2 * (3 + 4) * 5	70	70	Pass

No.	Expression	Expected	Program result	Status
6	10 / 4	2.5	2.5	Pass
7	2 * -3	-6	-6	Pass
8	-(2 + 3) * 4	-20	-20	Pass
9	5 / 0	Error message	Error: division by zero	Pass
10	(2 + 3	Error message	Error: invalid expression	Pass
11	2 +	Error message	Error: invalid expression	Pass
12	abc	Error message	Error: invalid expression	Pass

4.4 Discussion of Results

All tests produced the expected results. Test 4 shows that operators of equal precedence are evaluated from left to right (7 - 2 - 1 = 4, not 6). Tests 1 to 3 show that multiplication is done before addition and subtraction and that brackets are solved first. Tests 9 to 12 show that wrong input does not crash the program: an error message is written for that line and the program continues with the next expression. The separator lines in output.txt are identical to those in input.txt, as required.

4.5 Assumptions and Limitations

- A separator is any line made only of three or more dashes.
- Each expression is written on one line.
- Only the operators +, -, * and / and round brackets are supported (no powers or functions).
- Results use floating-point numbers, so very long decimals are rounded to four places.

5. Conclusion

This assignment showed how a simple data structure, the stack, can solve a real problem. By using one stack for numbers and another for operators, the program correctly handles operator precedence, brackets, negative numbers and decimals, and it reads from and writes to text files in the required format. Testing confirmed that the program gives correct answers and handles errors safely.

While completing the assignment, the Last-In, First-Out principle, the cost of stack operations and the importance of testing with both normal and invalid input were understood. In future the program could be extended with powers, functions or a graphical interface.

The complete project (Python source code, input.txt, output.txt and README.md) was pushed to GitHub and the lecturer was added as a collaborator.

GitHub repository: [Paste your repository link here]

6. References

1. Python Software Foundation. (2026). Python 3 Documentation: Data Structures. https://docs.python.org/3/tutorial/datastructures.html
2. Python Software Foundation. (2026). Python 3 Documentation: Reading and Writing Files. https://docs.python.org/3/tutorial/inputoutput.html
3. Dijkstra, E. W. (1961). Algol 60 translation: Shunting-Yard algorithm. Mathematisch Centrum, Amsterdam.
4. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.
5. GitHub Docs. (2026). Inviting collaborators to a personal repository. https://docs.github.com
6. Lecture notes: DCOMP320 Data Structures and Algorithms, Limkokwing University of Creative Technology, Sierra Leone.

7. Appendix A: Source Code (stack_evaluator.py)

```
"""
DCOMP320 - Data Structures and Algorithms
Assignment: Stacks Implementation

Reads math expressions from input.txt, evaluates each one using
stacks, and writes the results to output.txt (same format as input).

Run:  python stack_evaluator.py
      python stack_evaluator.py my_input.txt my_output.txt
"""

import sys


# ---------------------------------------------------------------
# PART 1: The Stack class (push, pop, peek + helpers)
# ---------------------------------------------------------------
class Stack:
    """A simple stack: Last-In, First-Out (LIFO)."""

    def __init__(self):
        self._items = []          # a Python list stores our items

    def push(self, item):
        """Put an item on top of the stack."""
        self._items.append(item)

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        """Look at the top item WITHOUT removing it."""
        if self.is_empty():
            raise IndexError("peek at empty stack")
```

```
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)


# ---------------------------------------------------------------
# PART 2: Parsing and evaluating an expression
# ---------------------------------------------------------------
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/": 2}


def tokenize(expression):
    """
    Break a string like '(8 / 4) + 7' into pieces (tokens):
    ['(', 8.0, '/', 4.0, ')', '+', 7.0]
    Also handles decimals and negative numbers like -5 or 2 * -3.
    """
    tokens = []
    i = 0
    n = len(expression)

    while i < n:
        ch = expression[i]

        if ch.isspace():                    # ignore spaces
            i += 1

        elif ch.isdigit() or ch == ".":     # read a whole number
            start = i
            while i < n and (expression[i].isdigit() or expression[i] == "."):
                i += 1
            tokens.append(float(expression[start:i]))

        elif ch in "+-*/":
```

```
            # A '-' is "unary" (a negative sign) if it comes at the start,
            # or right after another operator or an opening bracket.
            is_unary = ch == "-" and (
                not tokens or tokens[-1] in PRECEDENCE or tokens[-1] == "("
            )
            if is_unary:
                j = i + 1
                while j < n and expression[j].isspace():
                    j += 1
                if j < n and (expression[j].isdigit() or expression[j] == "."):
                    start = j                  # -5  -> the number -5
                    while j < n and (expression[j].isdigit() or expression[j] == "."):
                        j += 1
                    tokens.append(-float(expression[start:j]))
                    i = j
                else:                          # -(...) -> -1 * (...)
                    tokens.append(-1.0)
                    tokens.append("*")
                    i += 1
            else:
                tokens.append(ch)
                i += 1

        elif ch in "()":
            tokens.append(ch)
            i += 1

        else:
            raise ValueError(f"invalid character '{ch}'")

    return tokens


def apply_operator(operands, operators):
    """Take one operator and two numbers off the stacks, calculate, push result."""
    op = operators.pop()
    right = operands.pop()      # the second number comes off first
    left = operands.pop()
```

```
    if op == "+":
        operands.push(left + right)
    elif op == "-":
        operands.push(left - right)
    elif op == "*":
        operands.push(left * right)
    elif op == "/":
        if right == 0:
            raise ZeroDivisionError("division by zero")
        operands.push(left / right)


def evaluate(expression):
    """
    Evaluate an expression using TWO stacks:
      - operands  : holds the numbers
      - operators : holds + - * / and ( brackets
    This is a simplified version of Dijkstra's Shunting-Yard idea.
    """
    operands = Stack()
    operators = Stack()

    for token in tokenize(expression):

        if isinstance(token, float):                # a number
            operands.push(token)

        elif token == "(":
            operators.push(token)

        elif token == ")":
            # solve everything inside the brackets
            while not operators.is_empty() and operators.peek() != "(":
                apply_operator(operands, operators)
            if operators.is_empty():
                raise ValueError("mismatched parentheses")
            operators.pop()                         # remove the '('
```

```
        else:                               # + - * /
            # Before pushing, finish any earlier operator that must go first
            while (not operators.is_empty()
                   and operators.peek() != "("
                   and PRECEDENCE[operators.peek()] >= PRECEDENCE[token]):
                apply_operator(operands, operators)
            operators.push(token)

    # finish whatever is left
    while not operators.is_empty():
        if operators.peek() == "(":
            raise ValueError("mismatched parentheses")
        apply_operator(operands, operators)

    result = operands.pop()
    if not operands.is_empty():
        raise ValueError("invalid expression")
    return result


def format_result(value):
    """Show 13.0 as 13, and 2.5 as 2.5."""
    if value == int(value):
        return str(int(value))
    return str(round(value, 4))


def safe_evaluate(expression):
    """Evaluate, but return a friendly message instead of crashing."""
    try:
        return format_result(evaluate(expression))
    except ZeroDivisionError:
        return "Error: division by zero"
    except (ValueError, IndexError):
        return "Error: invalid expression"
```

```
# ---------------------------------------------------------------
# PART 3: Reading input.txt and writing output.txt
# ---------------------------------------------------------------
def is_separator(line):
    """A separator is a line made only of dashes, e.g. -----"""
    return len(line) >= 3 and set(line) == {"-"}


def process_file(input_path, output_path):
    output_lines = []

    with open(input_path, "r") as infile:
        for raw_line in infile:
            line = raw_line.strip()
            if line == "":
                continue                    # skip blank lines
            if is_separator(line):
                output_lines.append(line)       # keep the same separator
            else:
                output_lines.append(safe_evaluate(line))

    with open(output_path, "w") as outfile:
        outfile.write("\n".join(output_lines) + "\n")

    return output_lines


def main():
    input_path = sys.argv[1] if len(sys.argv) > 1 else "input.txt"
    output_path = sys.argv[2] if len(sys.argv) > 2 else "output.txt"

    try:
        results = process_file(input_path, output_path)
    except FileNotFoundError:
        print(f"Could not find '{input_path}'. Put it in the same folder.")
        return

    print(f"Done! {len(results)} lines written to {output_path}")
```

```
if __name__ == "__main__":
    main()
```